STA 5635 HW 4

Palmer Swanson

February 4, 2026

Problem 1

TISP variable selection method ith threshold operator:

$$\Theta(x, \lambda, \eta) = \begin{cases} 0 & \text{if} |x| \leq \lambda \\ \frac{x}{1+\eta} & |x| > \lambda \end{cases}$$

Penalized logistic regression:

$$L(\boldsymbol{\omega}) = \frac{1}{N} \sum_{j=1}^{N} \ln(1 + e^{-y_j^* \boldsymbol{\omega}^T \boldsymbol{x}_j}) + \lambda \sum_{i=1}^{p} p(\omega_i)$$

assuming $y_j^* \in \{-1, 1\}$ and algorithm iteration:

$$\boldsymbol{\omega}^{t+1} = \Theta\left(\boldsymbol{\omega}^t + \eta' X^T \left[\boldsymbol{y} - \frac{1}{1 + e^{-X\boldsymbol{\omega}^t}}\right], \lambda\right)$$

assuming $y_j \in \{-0, 1\}$.

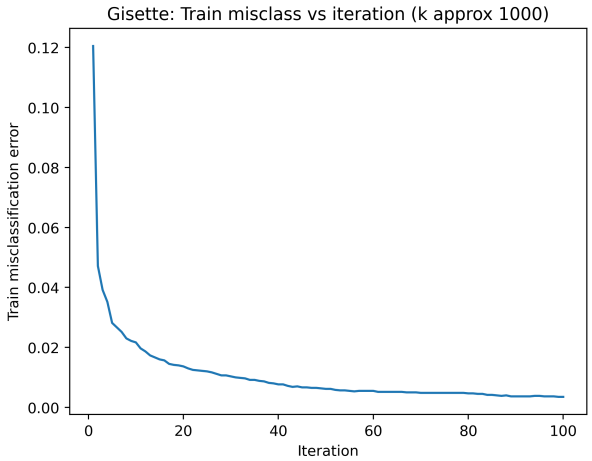
For $\eta = 0$ and 100 iterations, the first portion of question 1 is shown in Figure 1. We search for appropiate λ by performing a grid search across a range of values and selected the value that is closest to the desired number of features.

Second portion of question 1 is hown in Figure 2

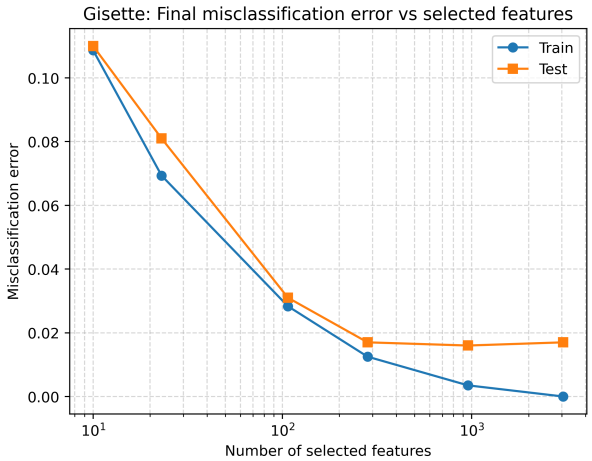
The table reporting the train and test misclassification error, the test AUC, corresponding number of features selected, values of λ, and the training time are reported in Table 1.

Dataset	Target Features	Selected Features	λ	Train Misclass.	Test Misclass.	Test AUC	Train Time (sec)
Gisette	10	10	0.196304	0.108667	0.110000	0.942920	1.844992
Gisette	30	23	0.138489	0.069333	0.081000	0.972464	1.824993
Gisette	100	107	0.086975	0.028333	0.031000	0.993964	1.795040
Gisette	300	282	0.054623	0.012500	0.017000	0.997420	1.805995
Gisette	1000	956	0.019179	0.003500	0.016000	0.998280	2.836653
Gisette	3000	3048	0.004229	0.000000	0.017000	0.997964	1.776995
Dexter	10	12	0.138489	0.116667	0.166667	0.932844	0.377510
Dexter	30	32	0.097701	0.046667	0.116667	0.959911	0.404991
Dexter	100	117	0.068926	0.003333	0.073333	0.975733	0.373484
Dexter	300	263	0.054623	0.000000	0.070000	0.979600	0.380031
Dexter	1000	1210	0.038535	0.000000	0.083333	0.968400	0.640966
Dexter	3000	1739	0.030539	0.000000	0.096667	0.966978	0.398962

Table 1: Train and Test Misclassification Errors, Test AUC, Number of Selected Features, values of λ, and training time, TISP algorithm.



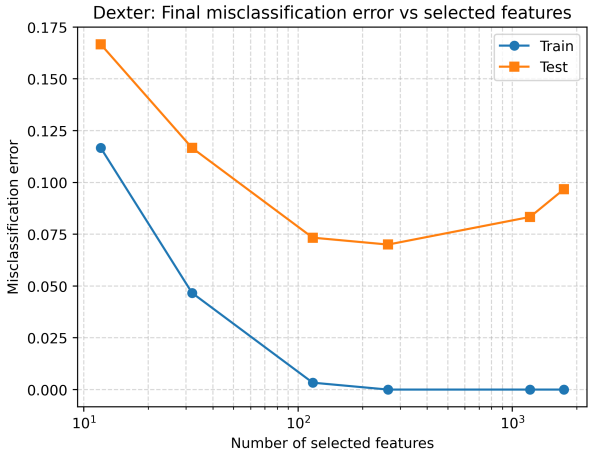
(a) Training misclassification error vs Iteration Number



(b) Train and Test Misclassification error vs Number of Selected Features

Figure 1: Train Misclassification Error vs. TISP iteration (left) and Train and Test Misclassification Error vs. Number of Selected Features (right). Gisette Dataset.

(a) Training misclassification error vs Iteration Number



(b) Train and Test Misclassification error vs Number of Selected Features

Figure 2: Train Misclassification Error vs. TISP iteration (left) and Train and Test Misclassification Error vs. Number of Selected Features (right). Dexter Dataset.

Problem 2

FSA variable selection using logistic loss (w/o shrinkage, FSA doesn't use a shrinkage penalty):

$$L_D(\beta) = \frac{1}{N}\sum_{i=1}^{N} \ln(1 + e^{-y_i \beta^T \boldsymbol{x}_j})$$

We can compute the gradient for one element with a few key facts, and a substitution: $u = -y_i \beta^T \boldsymbol{x}_j$:

$$\frac{d}{du}\log(1+e^u) = \frac{e^u}{1+e^u} = \frac{1}{1+e^{-u}} \qquad \frac{d}{d\beta}u = -y_i \boldsymbol{x}_i$$

$$\frac{d}{d\beta}\log(1+e^u) = \frac{1}{1+e^{-u}}(-y_i \boldsymbol{x}_i)$$

substitution gives us:

$$\frac{d}{d\beta}\log(1+e^{-y_i\beta^T\boldsymbol{x}_j}) = \frac{1}{1+e^{y_i\beta^T\boldsymbol{x}_j}}(-y_i \boldsymbol{x}_i)$$

With this, we can get the gradient for all N:

$$\nabla L_D(\beta) = \frac{-1}{N}\sum_{i=1}^{N} \frac{(y_i \boldsymbol{x}_i)}{1+e^{y_i\beta^T\boldsymbol{x}_j}}$$

We also use an inverse scheduler with parameter μ:

$$M_i = k + (p-k)\max\left(0, \frac{N^{iter} - 2i}{2i\mu + N^{iter}}\right)$$

For $s = 0.001$, $\mu = 100$, and 100 iterations, the first portion of question 2 is shown in Figure 3:

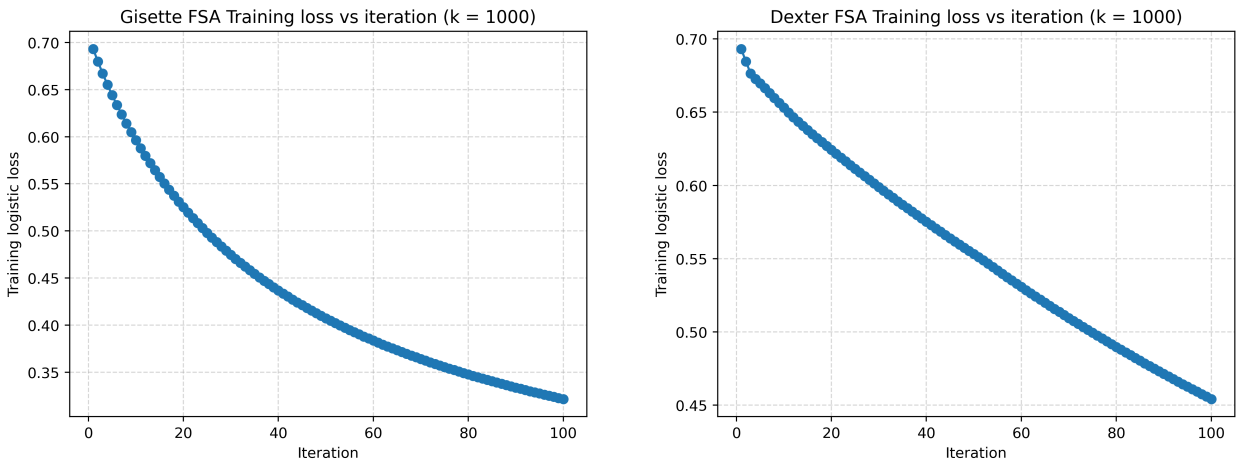
At first glance, neither datasets training loss converges, which makes sense. μ is only 100 and we only have 100 iterations; μ is usually 300 and we typically have more iterations.

The table reporting train and test misclassification errors, test AUC, and training time is shown in Table 2:

With only two exceptions (the 1000 k case), TISP appears to train faster. Specifically, the ratio of the computation times FSA/TISP is greater than 1 for all but the 1000 feature case in both datasets. Did not include this table to abide by submission guidelines.

We can also include the train/test misclassification error rate vs. k for the two datasets in Figure 4:

The two methods appear to be quit similar, with only slight deviations in trajectories.



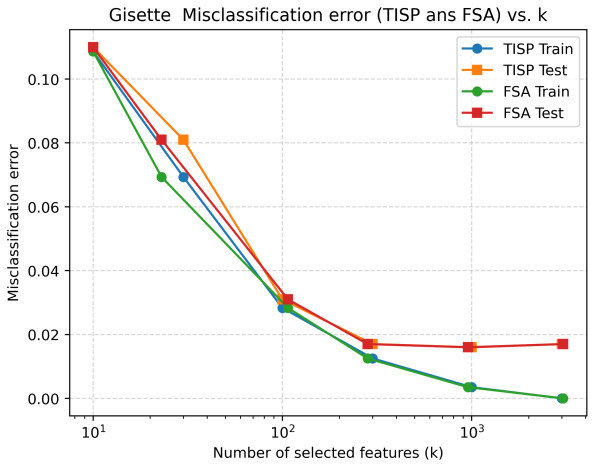
(a) Gisette Dataset

(b) Dexter Dataset

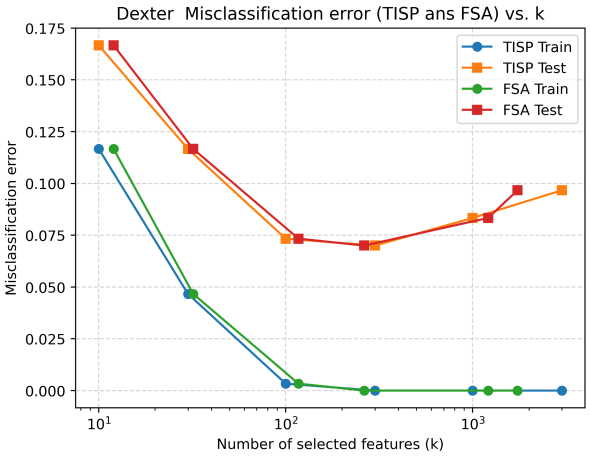
Figure 3: Plot of Training Loss vs. Iteration Number for $k = 1000$. Gisette Dataset (left), Dexter Dataset (right).

Dataset	Target k	Selected	Train Miscl.	Test Miscl.	Test AUC	Time (s)
Gisette	10	10	0.139000	0.149000	0.917434	1.867002
Gisette	30	30	0.143333	0.160000	0.928632	1.901725
Gisette	100	100	0.143000	0.156000	0.935600	1.899999
Gisette	300	300	0.112167	0.112000	0.960868	1.897630
Gisette	1000	1000	0.075833	0.072000	0.979972	1.881005
Gisette	3000	3000	0.062500	0.062000	0.984572	1.900006
Dexter	10	10	0.280000	0.290000	0.917489	0.470007
Dexter	30	30	0.173333	0.193333	0.936178	0.476007
Dexter	100	100	0.056667	0.106667	0.972356	0.459002
Dexter	300	300	0.013333	0.090000	0.975378	0.475998
Dexter	1000	1000	0.003333	0.080000	0.972667	0.451998
Dexter	3000	3000	0.000000	0.123333	0.938000	0.448999

Table 2: Train and Test Misclassification Errors, Test AUC, and training time, FSA algorithm.



(a) Gisette Dataset



(b) Dexter Dataset

Figure 4: Plot of Training Loss vs. Target Features. Gisette Dataset (left), Dexter Dataset (right).

Code

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
import time
import matplotlib.pyplot as plt
from sklearn.metrics import roc_auc_score, roc_curve

#Q1 datasets
q1_datasets = [
    {
        "name": "Gisette",
        "X_train": "assignments/hw04/data/gisette_train.data",
        "y_train": "assignments/hw04/data/gisette_train.labels",
        "X_valid": "assignments/hw04/data/gisette_valid.data",
        "y_valid": "assignments/hw04/data/gisette_valid.labels",
        "missclass_1000_out":
   "assignments/hw04/figures/gisette_misclass_1000.png",
        "FSA_out": "assignments/hw04/figures/gisette_misclass_1000_FSA.png"
    },
    {
        "name": "Dexter",
        "X_train": "assignments/hw04/data/dexter_train.csv",
        "y_train": "assignments/hw04/data/dexter_train.labels",
        "X_valid": "assignments/hw04/data/dexter_valid.csv",
        "y_valid": "assignments/hw04/data/dexter_valid.labels",
```

```
        "missclass_1000_out":
    "assignments/hw04/figures/dexter_misclass_1000.png",
        "FSA_out": "assignments/hw04/figures/dexter_misclass_1000_FSA.png"
    }
]

#thresholding function, w is vector of length p+1
#I don't think we should penalize the intercept
#https://stats.stackexchange.com/questions/559416/ridge-regression-subtlety-on-intercept
#will element wise decide whether or not to select variable
def threshold_operator(ws, lamb):
     #make copy of w vector
     w_cop = ws.copy()

     #https://numpy.org/devdocs/reference/generated/numpy.ones_like.html
     #plus don't do anything to the intercept
     #We should just get a boolean vector (F, T, T, T, T, etc)
     mask = np.ones_like(w_cop, dtype=bool)
     mask[0] = False

     #threshold  booleanwise
     w_cop[mask & (np.abs(w_cop) <= lamb)] = 0.0

     return w_cop


#Q1 parameters
iters = 100
eta = 0.0
features = [10, 30, 100, 300, 1000, 3000]

#we're going to be using logspace anyways
lambdas_to_search = np.logspace(-5, 0, 100)

results = []
#Q1
for d in q1_datasets:
    #get name of dataset
    name = d["name"]
    print(name)

    #Load datasets.  use default read.csv if Dexter dataset
    if name == "Dexter":
        X_train = pd.read_csv(d["X_train"], header=None)
        X_valid = pd.read_csv(d["X_valid"], header=None)
    else:
        X_train = pd.read_csv(d["X_train"], delim_whitespace=True, header=None)
```

```
        X_valid = pd.read_csv(d["X_valid"], delim_whitespace=True, header=None)

    y_train = pd.read_csv(d["y_train"], header=None).values.ravel()
    y_valid = pd.read_csv(d["y_valid"], header=None).values.ravel()

    #convert labels to 0/1 for later
    y_train_converted_01 = (y_train == 1).astype(int)
    y_valid_converted_01 = (y_valid == 1).astype(int)

    #standardize
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_valid)

    #add column of 1s to augment w
    X_train_tilde = np.column_stack([np.ones(len(X_train_scaled)),
    X_train_scaled])
    X_valid_tilde = np.column_stack([np.ones(len(X_test_scaled)), X_test_scaled])

    #get N, p
    N, p = X_train_tilde.shape

    #from notes, eta' can be 1/N
    eta_prime = 1.0/N

    #ok so what are we doing here.  We can to find a lambda that gets us the
    closest to some number
    #of features.  first thing we need to do is loop over the number of possible
    features.
    #going to be at least 2 loops
    #basically a grid search over the possible lambda values
    for k in features:
        print(k)
        best_lambda = None
        best_delta = None
        best_selected_vars = None

        #this loops just searches for the best lambda
        for j in lambdas_to_search:
            #print(j)

            #initialize w to 0s
            w = np.zeros(p)

            #loop over the number of iterations
            for i in range(iters):
                #get predictor
```

```
115                 z = X_train_tilde @ w
116
117                 #build fractional part
118                 frac = 1.0/(1.0 + np.exp(-z))
119
120                 #build y- fractional part
121                 difference = y_train_converted_01 - frac
122
123                 #get the update portion
124                 update_port = X_train_tilde.T @ difference
125
126                 #temporary new omega and do the thresholding
127                 temp = w + (eta_prime*update_port)
128                 w = threshold_operator(temp, j)
129
130             #need to count how many variables were selected
131             #don't count itnercept and give a small tolerance
132             #check to see if we are close to the desired number of features
133             k_selected = int(np.sum(np.abs(w[1:]) > 1e-10))
134             delta = abs(k_selected - k)
135
136             #if we don't have a best distance, of if the gap is smaller than the
    previous
137             #best, replace gap, lambda, number of variables selected
138             if (best_delta is None) or (delta < best_delta):
139                 best_delta = delta
140                 best_lambda = j
141                 best_selected_vars = k_selected
142
143         #ok so now we have the lambdas.  Use them to run this again
144         #now loop over everything again to time plot and get misclass rates
145         w = np.zeros(p)
146
147         #need to store missclass error only when trying to select 1000 features
148         misclass_1000 = []
149
150         start = time.time()
151         for t in range(iters):
152             z = X_train_tilde @ w
153             frac = 1.0 / (1.0 + np.exp(-z))
154             difference = (y_train_converted_01 - frac)
155             update_port = X_train_tilde.T @ difference
156             temp = w + (eta_prime*update_port)
157             w = threshold_operator(temp, best_lambda)
158
159             #need to track within this iterations loop
160             #now how to do that
```

```
            if k == 1000:
                z_new = X_train_tilde @ w
                p_train_1 = np.exp(-np.logaddexp(0.0, -z_new))
                p_train_0 = np.exp(-np.logaddexp(0.0, z_new))
                odds_train = p_train_1/p_train_0
                yhat_train = (odds_train > 1.0).astype(int)
                train_err = np.mean(yhat_train != y_train_converted_01)

                #append to misclass_1000
                misclass_1000.append(train_err)

        train_time = time.time() - start

        k_selected = int(np.sum(np.abs(w[1:]) > 1e-10))

        #train and test misclass errors
        z_train = X_train_tilde @ w
        z_test = X_valid_tilde @ w

        #get probabilities for 1 and 0
        p_train_1 = np.exp(-np.logaddexp(0.0, -z_train))
        p_train_0 = np.exp(-np.logaddexp(0.0, z_train))

        p_test_1 = np.exp(-np.logaddexp(0.0, -z_test))
        p_test_0 = np.exp(-np.logaddexp(0.0, z_test))

        #compute the odds of train and test
        odds_train = p_train_1/p_train_0
        odds_test = p_test_1/p_test_0

        #make predictions.  Predcit 1 if Odds > 1
        #as.int makes things nicer
        yhat_train = (odds_train > 1.0).astype(int)
        yhat_test = (odds_test > 1.0).astype(int)

        #misclassification error
        train_err = np.mean(yhat_train != y_train_converted_01)
        test_err = np.mean(yhat_test != y_valid_converted_01)

        #compute AUC
        test_auc = roc_auc_score(y_valid_converted_01, X_valid_tilde@w)

        #plotting the misclass_error vs iterations for 1000 features
        if k == 1000:
            plt.figure()
            plt.plot(np.arange(1, iters + 1), misclass_1000)
            plt.xlabel("Iteration")
```

```
            plt.ylabel("Train misclassification error")
            plt.title(f"{name}: Train misclass vs iteration (k approx 1000)")
            plt.savefig(d["missclass_1000_out"], dpi=400, bbox_inches="tight")
            plt.close()
            #plt.show()

        #append to results
        results.append({
            "dataset": name,
            "target_features": k,
            "selected_features": k_selected,
            "selected_lambda": best_lambda,
            "train_misclass": train_err,
            "test_misclass": test_err,
            "test_auc": test_auc,
            "train_time_sec": train_time
        })

#df for both Gisette and Dexter
df_q1 = pd.DataFrame(results)

#rename columns.  LATEX wasn't liking the column names
df_q1_latex = df_q1.rename(columns={
    "dataset": "Dataset",
    "target_features": "Target Features",
    "selected_features": "Selected Features",
    "selected_lambda": r"$\lambda$",
    "train_misclass": "Train Misclass.",
    "test_misclass": "Test Misclass.",
    "test_auc": "Test AUC",
    "train_time_sec": "Train Time (sec)"
})

#need to send this table to LATEX
LATEX_table = df_q1_latex.to_latex(
    index=False,
    caption=None,
    label=None,
    column_format="c" * df_q1_latex.shape[1],
    escape=False
)

#save latex table
with open("assignments/hw04/output/misclass_table.tex", "w") as f:
    f.write(LATEX_table)

#plot the final train and test misclassification error vs # of selected features
```

```
#using a semilog axis
for name in df_q1["dataset"].unique():
    #create filename to save plot
    outpath = f"assignments/hw04/figures/{name}_misclass_vs_features.png"

    #select the results for a given dataset
    sub = df_q1[df_q1["dataset"] == name].copy()

    #works better sorted
    #sub = sub.sort_values("selected_features")

    plt.figure()
    plt.semilogx(sub["selected_features"].to_numpy(),
    sub["train_misclass"].to_numpy(), marker="o", label="Train")
    plt.semilogx(sub["selected_features"].to_numpy(),
    sub["test_misclass"].to_numpy(), marker="s", label="Test")

    plt.xlabel("Number␣of␣selected␣features")
    plt.ylabel("Misclassification␣error")
    plt.title(f"{name}:␣Final␣misclassification␣error␣vs␣selected␣features")
    plt.legend()
    plt.grid(True, which="both", linestyle="--", alpha=0.5)
    plt.savefig(outpath, dpi=400, bbox_inches="tight")
    plt.close()
    #plt.show()


#Q2, using same datasets
q2_datasets = q1_datasets

s = 0.001
mu = 100
iters = 100
features = [10, 30, 100, 300, 1000, 3000]

results = []

for d in q2_datasets:
     #get name of dataset
    name = d["name"]
    print(name)

    #Load datasets.  use default read.csv if Dexter dataset
    if name == "Dexter":
        X_train = pd.read_csv(d["X_train"], header=None)
        X_valid = pd.read_csv(d["X_valid"], header=None)
    else:
```

```
        X_train = pd.read_csv(d["X_train"], delim_whitespace=True, header=None)
        X_valid = pd.read_csv(d["X_valid"], delim_whitespace=True, header=None)

    #need to check that y is +1 or -1.  If loop?
    y_train = pd.read_csv(d["y_train"], header=None).values.ravel()
    y_valid = pd.read_csv(d["y_valid"], header=None).values.ravel()

    #need to force these ya values to be -1/+1
    #https://numpy.org/devdocs/reference/generated/numpy.where.html
    y_train = np.where(y_train > 0, 1.0, -1.0)
    y_valid = np.where(y_valid > 0, 1.0, -1.0)

    #convert labels to 0/1 for likelihood to work with AUC
    y_train_converted_01 = (y_train == 1).astype(int)
    y_valid_converted_01 = (y_valid == 1).astype(int)

    #standardize, not forgetting this part this time
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_valid)

    #add column of 1s to augment w
    X_train_tilde = np.column_stack([np.ones(len(X_train_scaled)),
    X_train_scaled])
    X_valid_tilde = np.column_stack([np.ones(len(X_test_scaled)), X_test_scaled])

    #get N, p
    N, p = X_train_tilde.shape

    training_loss_1000 = []

    #loop over the number of possible features
    for feat in features:
        #initialize to 0
        beta = np.zeros(p)

        start = time.time()

        for i in range(iters):
            #get predictor
            z = X_train_tilde @ beta
            yz = y_train * z

            #need numpy function I found
            #https://numpy.org/devdocs/reference/generated/numpy.log1p.html
            exponent_part = np.exp(-yz)
            log_part = np.log1p(exponent_part)
```

```
346             loss = np.mean(log_part)
347 
348             #keep track of the 1000 feature case
349             if feat == 1000:
350                 training_loss_1000.append(loss)
351 
352             #no sparsity penalty
353             #compute gradient.  Per quiz and notes, FSA does NOT use a sparsity
    penalty for feature
354             #selection
355             part1 = 1.0 / (1.0 + np.exp(yz))
356             grad = -(1/N)*(X_train_tilde.T @ (y_train*part1))
357 
358             #update Beta using gradient step
359             #is s equivalent to eta in the notes??  It  has to be
360             beta = beta - (s*grad)
361 
362             #inverse scheduler
363             denom = (2.0*i*mu) + iters
364             numer = (iters - (2*i))
365             ratio = numer/denom
366 
367             #get maximum of 0 or the ratio.  Exclude the intercept
368             p_feat = p-1
369             max_portion = max(0, ratio)
370             Mi = np.round(feat + (p_feat-feat)*max_portion)
371             Mi = int(Mi)
372 
373             #need to keep the top Mi variables, not including intercept
374             #sort.  Keep the largest Mi.
375             #beta length p.  beta[0] = intercept.  beta[1:] feature coefficients
376             #Mi = # of features permitted at this iteration
377             #https://numpy.org/doc/stable/reference/generated/numpy.argsort.html
378             b = beta[1:]
379             abs_b = abs(b)
380 
381             #first entries in sort will be the smallest.  Need to set all but
    the last
382             #Mi to 0 aka we need to set the first p_feat-Mi to 0
383             sort = np.argsort(abs_b)
384             b[sort[:-Mi]] = 0.0
385             beta[1:] = b
386             #should end up with something that looks like
387             #beta = [1.0, 0.0, 3.0, 5.0, -3.0, 0.0, 0.0, 0.0,...]
388 
389         end = time.time()
390         run_time = end-start
```

```
391
392         #gotta be a better way to do this stuff
393         #need to make predictions now for misclass and AUC
394         z_train = X_train_tilde @ beta
395         z_test = X_valid_tilde @ beta
396
397         #get probabilities for 1 and 0
398         p_train_1 = np.exp(-np.logaddexp(0.0, -z_train))
399         p_train_0 = np.exp(-np.logaddexp(0.0, z_train))
400
401         p_test_1 = np.exp(-np.logaddexp(0.0, -z_test))
402         p_test_0 = np.exp(-np.logaddexp(0.0, z_test))
403
404         #compute the odds of train and test
405         odds_train = p_train_1/p_train_0
406         odds_test = p_test_1/p_test_0
407
408         #make predictions.  Predcit 1 if Odds > 1
409         #as.int makes things nicer
410         yhat_train = (odds_train > 1.0).astype(int)
411         yhat_test = (odds_test > 1.0).astype(int)
412
413         #misclassification error
414         train_err = np.mean(yhat_train != y_train_converted_01)
415         test_err = np.mean(yhat_test != y_valid_converted_01)
416
417         #compute AUC
418         test_auc = roc_auc_score(y_valid_converted_01, X_valid_tilde@beta)
419
420         #compute how many variables are non 0 aka selected
421         selected = int(np.sum(beta[1:] != 0))
422         print(selected)
423
424         #append to results
425         results.append({
426             "dataset": name,
427             "target_features": feat,
428             "selected_features": selected,
429             "train_misclass": train_err,
430             "test_misclass": test_err,
431             "test_auc": test_auc,
432             "train_time_sec": run_time
433         })
434
435     #plot the training loss vs iteration number
436     plt.figure()
```

```
    plt.plot(range(1, len(training_loss_1000) + 1), training_loss_1000,
    marker="o")
    plt.xlabel("Iteration")
    plt.ylabel("Training␣logistic␣loss")
    plt.title(f"{name}␣FSA␣Training␣loss␣vs␣iteration␣(k␣=␣1000)")
    plt.grid(True, linestyle="--", alpha=0.5)
    plt.savefig(d["FSA_out"], dpi=400, bbox_inches="tight")
    plt.close()
    #plt.show()

#df for both Gisette and Dexter
df_q2 = pd.DataFrame(results)

#renaming this table for LATEX
df_q2_latex = df_q2.rename(columns={
    "dataset": "Dataset",
    "target_features": "Target␣$k$",
    "selected_features": "Selected",
    "train_misclass": "Train␣Misclass.",
    "test_misclass": "Test␣Misclass.",
    "test_auc": "Test␣AUC",
    "train_time_sec": "Time␣(s)"
})

#need to send this table to LATEX
LATEX_table = df_q2_latex.to_latex(
    index=False,
    caption=None,
    label=None,
    column_format="c" * df_q2_latex.shape[1],
    escape=False
)

#save latex table
with open("assignments/hw04/output/misclass_table_q2.tex", "w") as f:
    f.write(LATEX_table)

#need to plot the train and test errors vs k
for name in df_q2["dataset"].unique():
    outpath = f"assignments/hw04/figures/{name}_misclass_TISP_vs_FSA_.png"
    #make copy of the df
    sub1 = df_q1[df_q1["dataset"] == name].copy()
    sub2 = df_q2[df_q2["dataset"] == name].copy()

    plt.figure()
    #TISP
```

```
    plt.semilogx(sub2["selected_features"].to_numpy(),
    sub1["train_misclass"].to_numpy(), marker="o", label="TISP␣Train")
    plt.semilogx(sub2["selected_features"].to_numpy(),
    sub1["test_misclass"].to_numpy(), marker="s", label="TISP␣Test")
    #FSA
    plt.semilogx(sub1["selected_features"].to_numpy(),
    sub1["train_misclass"].to_numpy(), marker="o", label="FSA␣Train")
    plt.semilogx(sub1["selected_features"].to_numpy(),
    sub1["test_misclass"].to_numpy(), marker="s", label="FSA␣Test")

    plt.xlabel("Number␣of␣selected␣features␣(k)")
    plt.ylabel("Misclassification␣error")
    plt.title(f"{name}␣␣Misclassification␣error␣(TISP␣ans␣FSA)␣vs.␣k")
    plt.legend()
    plt.grid(True, linestyle="--", alpha=0.5)
    plt.savefig(outpath, dpi=400, bbox_inches="tight")
    plt.close()

#want to compare the computation time for the two methods.
#not included in the writeup, but I did it.
df_q1["method"] = "TISP"
df_q2["method"] = "FSA"
df_time = pd.concat([df_q1, df_q2], ignore_index=True)

#need to pivot.  Chaining this stuff together like the pipe operator in R
df_time = (
    df_time[["dataset", "target_features", "method", "train_time_sec"]]
        .pivot(
            index=["dataset", "target_features"],
            columns="method",
            values="train_time_sec"
        )
        .assign(
            FSA_over_TISP=lambda x: (x["FSA"] / x["TISP"]).round(2)
        )
        .reset_index()
)

#rename columns
df_time_latex = df_time.rename(columns={
    "dataset": "Dataset",
    "target_features": "Target␣$k$",
    "TISP": "TISP␣Time␣(s)",
    "FSA": "FSA␣Time␣(s)",
    "FSA_over_TISP": "FSA␣/␣TISP"
})

```

```
latex_table = df_time_latex.to_latex(
    index=False,
    escape=False,
    float_format="%.2f",
    column_format="c" * df_time_latex.shape[1]
)

with open("assignments/hw04/output/time_comparison.tex", "w") as f:
    f.write(latex_table)
```